EXPERIMENT 1

Single-Qubit Gates (X, Z, H) and Bloch Sphere

Cell 1

```
!pip install qiskit
!pip install qiskit-aer
!pip install pylatexenc
```

Cell 2

```
from qiskit import *
from qiskit_aer import *
from qiskit.visualization import *
```

Cell 3

```
qc = QuantumCircuit(1)
qc.x(0)
qc.z(0)
qc.h(0)
qc.draw('mpl')
```

Cell 4

```
backend = Aer.get_backend('statevector_simulator' )
result = backend.run(transpile(qc,backend),shots = 1000).result()
psi = result.get_statevector(qc)
plot_bloch_multivector(psi)
```

EXPERIMENT 2

Bell State Preparation and Quantum State Visualization

Cell 1

```
!pip install qiskit
!pip install qiskit-aer
!pip install pylatexenc
```

Cell 2

```
from qiskit import *
from qiskit_aer import *
from qiskit.visualization import *
```

Cell 3

```
#bell state
qc = QuantumCircuit(2,2)
qc.x(1)
qc.h(0)
qc.cx(0,1)
qc.measure([0,1],[0,1])
qc.draw('mpl')
```

Cell 4

```
simulator = AerSimulator()
result = simulator.run(transpile(qc,simulator),shots = 1024).result()
counts = result.get_counts()
print(counts)
plot_histogram(counts)
```

Cell 5

```
backend = Aer.get_backend('statevector_simulator')
psi = backend.run(transpile(qc,backend),shots = 1024).result().get_statevector()
print(psi)
```

Cell 6

```
plot_bloch_multivector(psi)
```

Cell 7

```
plot_state_city(psi)
```

Cell 8

```
plot_state_hinton(psi)
```

Cell 9

```
plot_state_paulivec(psi)
```

EXPERIMENT 3

Bell State Generation Using Hadamard and CNOT

Cell 1

```
!pip install qiskit
!pip install qiskit-aer
!pip install pylatexenc
```

Cell 2

```
from qiskit import *
from qiskit_aer import *
from qiskit.visualization import *
```

Cell 3

```
qc = QuantumCircuit(2,2)
qc.h(0)
qc.cx(0,1)
qc.measure([0,1],[0,1])
qc.draw('mpl')
```

Cell 4

```
simulator = AerSimulator()
compiled_circuit = transpile(qc,simulator)
result = simulator.run(transpile(compiled_circuit,simulator),shots = 1024).result()
counts = result.get_counts()
print(counts)
plot_histogram(counts)
```

Cell 5

```
backend = Aer.get_backend('statevector_simulator' )
psi = backend.run(transpile(qc,backend),shots = 1024).result().get_statevector()
print(psi)
```

Cell 6

```
plot_bloch_multivector(psi)
```

Cell 7

```
plot_bloch_vector(psi)
```

Cell 8

```
plot_state_city(psi)
```

Cell 9

```
plot_state_hinton(psi)
```

Cell 10

```
plot_state_qsphere(psi)
```

EXPERIMENT 4

Superdense Coding

Cell 1

```
!pip install qiskit
!pip install qiskit-aer
!pip install pylatexenc
```

Cell 2

```
from qiskit import *
from qiskit_aer import *
from qiskit.visualization import *
```

Cell 3

```
def create(qc,a,b):
    qc.h(a)
    qc.cx(a,b)
qc = QuantumCircuit(2,2)
create(qc,0,1)
qc.draw('mpl')
```

Cell 4

```
def encode(qc,qubit,msg):
    if msg == '00':
        pass
    elif msg == '10':
        qc.x(qubit)
    elif msg == '01':
        qc.z(qubit)
    elif msg == '11':
        qc.z(qubit)
        qc.x(qubit)
    else:
        print("error")
message = input("enter any one state : 00,01,10,11 : ")
encode(qc,0,message)
```

Cell 5

```
def decode(qc,a,b):
    qc.cx(a,b)
    qc.h(a)
decode(qc,0,1)
qc.measure_all()
qc.draw('mpl')
```

Cell 6

```
simulator = AerSimulator()
result = simulator.run(transpile(qc,simulator),shots = 1024).result()
counts = result.get_counts()
print(counts)
plot_histogram(counts)
```

Cell 7

```
# combined of all
for message in['00','01','10','11']:
    qc = QuantumCircuit(2,2)
    create(qc,0,1)
    encode(qc,0,message)
    decode(qc,0,1)
    qc.measure_all()
    simulator = AerSimulator()
    result = simulator.run(transpile(qc,simulator),shots = 1000).result()
    counts = result.get_counts()
    display(qc.draw('mpl'))
    display(plot_histogram(counts) )
```

EXPERIMENT 5

SWAP Gate Using Three CNOT Gates

Cell 1

```
!pip install qiskit
!pip install qiskit_aer
!pip install pylatexenc
```

Cell 2

```
from qiskit import *
from qiskit_aer import *
from qiskit.visualization import *
```

Cell 3

```
qc = QuantumCircuit(2,2)
qc.cx(0,1)
qc.cx(1,0)
qc.cx(0,1)
qc.measure([0,1],[0,1])
qc.draw('mpl')
```

Cell 4

```
simulator = AerSimulator()
result = simulator.run(transpile(qc,simulator),shots = 1024).result()
counts = result.get_counts()
print(counts)
plot_histogram(counts)
```

Cell 5

```
qc = QuantumCircuit(2,2)
qc.x(0)
qc.cx(0,1)
qc.cx(1,0)
qc.cx(0,1)
qc.measure([0,1],[1,0])
qc.draw('mpl')
```

Cell 6

```
simulator = AerSimulator()
result = simulator.run(transpile(qc,simulator),shots = 1000).result()
counts = result.get_counts()
print(counts)
plot_histogram(counts)
```

EXPERIMENT 6

Deutsch-Jozsa Algorithm: Constant and Balanced Functions

Cell 1

```
!pip install qiskit
!pip install qiskit_aer
!pip install pylatexenc
```

Cell 2

```
from qiskit import *
from qiskit_aer import *
from qiskit.visualization import *
```

Cell 3

```
# constant function
n = int(input("enter the number of qubits"))
qc = QuantumCircuit(n+1,n)
qc.x(n)
i = 0
while i < n+1:
    qc.h(i)
    i = i+1
qc.barrier()
qc.x(n)
qc.barrier()
j = 0
while j < n:
    qc.h(j)
    j = j + 1
qc.barrier()
qc.measure(range(n),range(n))
qc.draw('mpl')
```

Cell 4

```
simulator = AerSimulator()
result = simulator.run(transpile(qc,simulator),shots = 1024).result()
counts = result.get_counts()
print(counts)
plot_histogram(counts)
```

Cell 5

```
#balance
qc = QuantumCircuit(3,2)
qc.x(2)
qc.barrier()
qc.h(0)
qc.h(1)
qc.h(2)
qc.barrier()
qc.cx(0,2)
qc.cx(1,2)
qc.barrier()
qc.h(0)
qc.h(1)
qc.barrier()
qc.measure([0,1],[0,1])
qc.draw('mpl')
```

Cell 6

```
simulator = AerSimulator()
result = simulator.run(transpile(qc,simulator),shots = 1024).result()
counts = result.get_counts()
print(counts)
plot_histogram(counts)
```

EXPERIMENT 7

Two-Qubit Quantum Circuit Using Hadamard and CZ Gates

Code transcribed from the supplied Jupyter Notebook screenshot

CELL 1 - CHECK QISKIT VERSION

```
import qiskit
print(qiskit.version.get_version_info())
```

CELL 2 - IMPORTS

```
from IPython.display import display
from qiskit_aer import Aer
from qiskit.visualization import plot_distribution
from qiskit import QuantumCircuit,transpile
```

CELL 3 - INITIAL CIRCUIT

```
qc = QuantumCircuit(2,2)
qc.h(0)
qc.h(1)
qc.barrier()
```

CELL 4 - DRAW CIRCUIT

```
qc.draw()
```

CELL 5 - CZ GATE AND BARRIER

```
qc.cz(0,1)
qc.barrier()
```

CELL 6 - DRAW CIRCUIT

```
qc.draw()
```

CELL 7 - APPLY GATES

```
qc.h([0,1])
qc.cz([0,1])
qc.cz(0,1)
qc.h([0,1])
```

CELL 8 - MEASURE AND DRAW

```
qc.measure([0,1],[0,1])
print(qc.draw())
```

CELL 9 - SIMULATE AND PLOT DISTRIBUTION

```
from IPython.display import display
from qiskit_aer import Aer
from qiskit.visualization import plot_distribution
from qiskit import QuantumCircuit,transpile

simulator = Aer.get_backend('qasm_simulator')
result = simulator.run(transpile(qc,simulator),shots = 2**16).result()
counts = result.get_counts()
print(counts)
display(plot_distribution(counts))
```

EXPERIMENT 8

Shor's Algorithm: Integer Factorization of N = 15

```
import math

def find_order(base, number):
    power = 1
    order = 1

    while True:
        power = (power * base) % number

        if power == 1:
            return order

        order += 1


def factor_using_shor(number, base):
    common_factor = math.gcd(base, number)

    if common_factor != 1:
        return common_factor, number // common_factor

    order = find_order(base, number)

    if order % 2 != 0:
        return None

    value = pow(base, order // 2, number)

    factor_a = math.gcd(value - 1, number)
    factor_b = math.gcd(value + 1, number)

    if factor_a == 1 or factor_a == number:
        return None

    if factor_b == 1 or factor_b == number:
        return None

    return factor_a, factor_b


N = 15
a = 2

result = factor_using_shor(N, a)

print("Number to factor:", N)
print("Chosen value of a:", a)
print("Factors obtained:", result)
```